

Development Report

The player class is designed to describe the player. This class stores essential variables. This includes the size, position and the image. Inside the class player, there is a method used as “void display ();” this is used to show the player on the screen. The appearance of the player is then changed to PImage which is then downloaded as PNG and makes the player into a blue rocket. This shows the size and placement of the image on the screen.

```
class Player
{
    int x, y ; //1. Data stored
    PImage blueImage;

    Player(int x, int y, PImage image) //2. Constructor
    {
        this.x = 400;
        this.y = 400;
        this.blueImage = image;
    }

    void display()
    {
        fill(250, 0, 0); //3. Methods
        image(blueImage, this.x, this.y, 100, 100);
        //rectMode(CENTER);
        //ellipse(this.x,this.y ,50,50);
    }
}
```

The Obstacle class is presented to be similar to the class player but it introduces new methods such as void move (); inside this method, two types of behaviours have been set. One for the boundaries makes the enemies bounce off the walls and don't float away from the screen; this creates a “bounce effect”. Then the second behaviour is the specific direction you want the enemies to go. This is used to move the enemies in particular directions. For example, press the right arrow all enemies will move to the left.

```
void move()
{
    x += dx*6;
    y += dy*6;
    // Check boundaries and reverse direction if necessary
    if (x > width || x < 0)
    {
        dx *= -1; // Reverse direction in x
    }
    if (y > height || y < 0)
    {
        dy *= -1; // Reverse direction in y
    }
}

void moveDown()
{
    this.y += 1*speed;
}
void moveUp()
{
    this.y -= 1*speed;
}
void moveRight()
{
    this.x += 1*speed;
}
void moveLeft()
{
    this.x -= 1*speed;
}
```

Void keyPressed(); this method is used to show the placement and direction of where the enemies are going.

```
void keyPressed()
{
    for (int i = 0; i < enemy.length; i++)
    {
        if ( enemy[i] != null ) {
            if (key == CODED) {
                if (keyCode == UP) {
                    enemy[i].moveDown(); //move all obstacles down
                } else if (keyCode == RIGHT)
                {
                    enemy[i].moveLeft(); // move all obstacles left
                } else if (keyCode == DOWN )
                {
                    enemy[i].moveUp(); //move all obstacles right
                } else if (keyCode == LEFT)
                {
                    enemy[i].moveRight(); // move all obstacle right
                }
            }
        }
    }
}
```

```
enemy = new Obstacle[15]; // amount of enemies
```

To reduce the repetition and improve the way the game works a for loop is implemented. This for loop helps write specific movements for each obstacle. An array list is then used to store a collection of obstacles in a simpler version without complicating the code and adding unnecessary codes. In this game an array list of 15 enemies that move according to the specific behaviours that have been.

```
blue = loadImage("blue.png");  
//blue.resize(100,100);  
imageMode(CENTER);  
player = new Player(250, 250, blue);  
player.blueImage.resize(100, 100);  
  
ALIEN = loadImage("A.png"); //name of the image that is being uploaded  
ALIEN.resize(100, 100);  
imageMode(CENTER);  
enemy = new Obstacle[15]; // amount of enemies
```

The screenshot above shows how PImage is used.

```
int xPos = (int) random(0, width); //enemies go random location  
int yPos = (int) random(0, height);  
enemy[i] = new Obstacle (xPos, yPos, ALIEN);
```

The picture above shows how to make the enemies go to random locations on the screen but theft is still kept within the boundaries of the obstacle class.